KV Cache Engine
Reference Model API

LonghornSilicon LLM Inference Accelerator

Block 2 of 4 — Bit-Accurate C++ / C / Python Models

Version: 0.1 (first public draft)
Date: 2026-05-14
Status: Verified against RTL testbench vectors
Source: `sw/reference_model/`

Contents

1. Overview

Bit-accurate reference implementations of the KV cache engine in three languages, all verified against the same test vectors.

Block	C++ class	`extern "C"` API	Python class	Tests
KV Cache Engine	`lhsi::KVCacheEngine`	`lhsi_kv_*`	`KVCacheEngine`	120 Python + 64 C++

Top-level orientation for the compiler team: see `sw/README.md`. ISA specification: see `docs/isa/kv_cache_engine_isa.pdf`.

2. Building and Testing

```
make test-all       # build C++ tests, run them, run all Python tests
make test           # only the C++ tests
make test-py        # only the Python tests
make shared         # libkv_cache_engine_ref.so
make static         # libkv_cache_engine_ref.a
make clean
```

Requires Python 3.10+, NumPy, and a C++17 compiler (gcc-9+ or clang-10+). The C++ tests pick up the RTL test vectors from `rtl/tb/testvectors/*.hex`; the Makefile regenerates them from `analysis/gen_kv_testvectors.py` on first build if absent.

3. KV Cache Engine

3.1 Configuration

```
#include "kv_cache_engine_ref.hpp"

lhsi::KVCacheEngineInfo info;
info.vector_dim    = 64;     // power of two (WHT constraint)
info.pq_bits       = 3;      // Lloyd-Max quantization bits
info.qjl_bits      = 1;      // QJL projection bits
info.sram_depth    = 1024;
info.coord_width   = 16;     // fixed-point coordinate width
info.coord_frac    = 12;     // fractional bits
info.norm_width    = 16;     // norm storage width
info.rotation_seed = 42;     // deterministic LCG seed
info.validate();             // asserts on invalid combinations
```

3.2 C++ Class API

```
#include "kv_cache_engine_ref.hpp"

lhsi::KVCacheEngine engine;   // default config

// Compress a key vector
std::vector<int16_t> key_vec(64, /* ... */);
```

```
lhsi::CompressedKey ck = engine.compress_key(key_vec);

// Decompress back
std::vector<int16_t> reconstructed = engine.decompress_key(ck);

// Compress a value vector (no QJL)
lhsi::CompressedValue cv = engine.compress_value(val_vec);
std::vector<int16_t> val_recon = engine.decompress_value(cv);

// Store/load with SRAM address
engine.store_kv(/*addr=*/0, key_vec.data(), val_vec.data(), 64);
engine.load_k(0, key_out.data(), 64);
engine.load_v(0, val_out.data(), 64);
```

3.3 Sub-Operations (Pipeline Stage Access)

For compiler backends that need individual pipeline stages:

```
// Each method mirrors one RTL pipeline stage
uint32_t norm = engine.compute_norm(vec, dim);
engine.normalize(vec, norm, normalized, dim);
engine.rotate(normalized, rotated, dim);
engine.quantize_pq(rotated, indices, dim);
engine.dequantize_pq(indices, dequantized, dim);
engine.qjl_compress(residual, signs, &res_norm, dim);
engine.qjl_decompress(signs, res_norm, correction, dim);
engine.inverse_rotate(corrected, output, dim);
```

3.4 Plain C (`extern "C"` API)

```
#include "kv_cache_engine_ref.hpp"

// Create with default config
lhsi_kv_handle_t* h = lhsi_kv_create_default();

// Or with custom config
lhsi_kv_info_t info = { .vector_dim = 64, .pq_bits = 3, /* ... */ };
lhsi_kv_handle_t* h = lhsi_kv_create(&info);

// Query config
lhsi_kv_info_t out_info = lhsi_kv_info(h);

// Compress key
uint32_t norm, res_norm;
uint8_t indices[64], signs[64];
lhsi_kv_compress_key(h, vec, 64, &norm, indices, &res_norm, signs);

// Decompress key
int16_t output[64];
lhsi_kv_decompress_key(h, norm, indices, res_norm, signs, 64, output);

// Compress/decompress value (no QJL fields)
```

```
lhsi_kv_compress_value(h, vec, 64, &norm, indices);
lhsi_kv_decompress_value(h, norm, indices, 64, output);

// Store/load with SRAM
lhsi_kv_store(h, 0, key_vec, val_vec, 64);
lhsi_kv_load_k(h, 0, key_out, 64);
lhsi_kv_load_v(h, 0, val_out, 64);

lhsi_kv_destroy(h);
```

3.5 Python

```
from kv_cache_engine_ref import KVCacheEngine, KVCacheEngineInfo

engine = KVCacheEngine()

# Compress key
ck = engine.compress_key(key_vector)
reconstructed = engine.decompress_key(ck)

# Compress value
cv = engine.compress_value(val_vector)
val_recon = engine.decompress_value(cv)

# Store/load with SRAM
engine.store_kv(addr=0, key=key_vector, value=val_vector)
k_out = engine.load_k(addr=0)
v_out = engine.load_v(addr=0)

# Static method (no state needed)
ck = KVCacheEngine.decide_compress_key(key_vector)
```

4. Numerical Semantics (Frozen)

All arithmetic is **fixed-point integer**, matching the RTL exactly:

- **Coordinates**: `COORD_WIDTH`-bit signed two's complement, `COORD_FRAC` fractional bits (default: Q4.12).
- **Norms**: `NORM_WIDTH`-bit unsigned, `NORM_FRAC` fractional bits (default: Q8.8).
- **WHT butterfly**: additions widen by 1 bit per stage ($\log_2 D$ stages), then truncate back to coordinate width.
- **Quantization**: nearest-centroid with 3 comparators per coordinate. Centroids are Lloyd-Max optimal for $\mathcal{N}(0, 1/\sqrt{D})$.
- **QJL**: random ± 1 matrix from deterministic LCG seed. Inner product $\rightarrow$ sign extraction. Decompression scales by $\sqrt{\pi/2}$ (fixed-point constant).
- **Integer sqrt**: non-restoring algorithm, bit-serial from MSB.

If the model disagrees with the chip on any input, the model is wrong. Open an issue with the failing vector.

5. Compressed Data Structures

5.1 `CompressedKey`

Field	Type	Bits	Purpose
`norm`	`uint32_t`	16	L2 norm of original vector
`indices`	`uint8_t[D]`	$3D$	Lloyd-Max centroid indices (0–7)
`residual_norm`	`uint32_t`	16	L2 norm of quantization residual
`signs`	`uint8_t[D]`	D	QJL sign bits
Total (D=64)		**288**	3.56× compression

5.2 `CompressedValue`

Field	Type	Bits	Purpose
`norm`	`uint32_t`	16	L2 norm of original vector
`indices`	`uint8_t[D]`	$3D$	Lloyd-Max centroid indices (0–7)
Total (D=64)		**208**	4.92× compression

6. Verification Status

Test	Block / Aspect	Status
120 Python unit tests	Full round-trip, all paths	Pass
64 C++ unit tests	Full round-trip, all paths	Pass
Compress/decompress round-trip	K and V paths	Bit-exact
K/V asymmetry verification	QJL on K only	Pass
Streaming vs batch agreement	tick() vs compress_*()	Pass
`extern "C"` = C++ class	C API parity	Pass
RTL replay vectors	Hex-file bit-exact match	Pass
Compression ratio ¿= 3×	Canonical trace	Pass

7. Co-Design Context

This directory is the deliverable for compiler-team integration. The reference model provides the same API contract that the hardware will expose, enabling compiler backends to target the KV cache engine before silicon or FPGA prototypes exist.

Next integration steps:

1. Compiler targets Python/C++ reference models (Phase 0 — now)
2. AXI-Lite + AXI-Stream on Zynq UltraScale+ FPGA (Phase 1)

3. Multi-block FPGA project with precision controller (Phase 2)
4. TSMC 16FFC silicon (Phase 3, 2027+)